

Integrating WebGPU: Advancing Graphics Rendering in the Elements Framework

Bachelor Thesis



Author: Eleftherios Toupis (Student ID: 4814)

Supervisor: Prof. Papagiannakis George

Co-Supervisor: Dr. Manos Kamarianakis

Department of Computer Science
School of Sciences & Engineering
University of Crete
Heraklion, Greece

September 17, 2024

Abstract

This thesis presents the architecture and implementation of a modern rendering engine designed to leverage the capabilities of both OpenGL and WebGPU APIs. By supporting dual shading languages, GLSL and WGSL, the engine allows developers to exploit the strengths of both platforms while ensuring flexibility and performance. The forward and deferred rendering pipelines form the core of the rendering subsystem, enabling efficient handling of complex scenes with multiple lights and dynamic elements.

Key aspects of the rendering engine include the seamless integration of advanced shading techniques, such as normal mapping, parallax occlusion mapping, and shadow casting, along with post-processing effects like SSAO. The engine's modular design facilitates the easy addition of custom rendering subsystems, allowing for further extensions and specialized effects, such as screen-space reflections and anti-aliasing.

This work also details the transition to WGSL, outlining the differences and limitations compared to GLSL, and how these differences influence shader development. Through practical examples, such as textured meshes, lighting models, and environmental effects like skyboxes, the thesis demonstrates the engine's versatility and the process of extending its functionality. The result is a robust and adaptable rendering engine suitable for a wide range of real-time graphics applications.

Declaration of Authorship

I, Eleftherios Toupis, declare that this thesis titled, "Integrating WebGPU: Advancing Graphics Rendering in the Elements Framework", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature: Eleftherios Toupis

Date: September 17, 2024

Acknowledgements

I would like to express my sincere gratitude to my supervising professor, Prof. Papagiannakis George, for his continuous support and invaluable guidance throughout the course of this project.

I also extend my thanks to Dr. Manos Kamarianakis and John Petropoulos for their insightful advice and helpful instructions.

I would also like to thank Andrew from the GetIntoGameDev channel on YouTube for his helpful tutorials on WebGPU, which greatly contributed to my understanding of the subject.

Lastly, I am grateful to Joey de Vries from LearnOpenGL.com for the tutorials and examples of basic graphics techniques implemented in OpenGL, as well as for his guidance in helping me understand how they work.

Contents

1	Introduction and Overview	1
1.1	The Elements framework	1
1.2	Elements' OpenGL Rendering Overview	1
1.3	What is WebGPU	3
1.4	Programming Languages (Python, WGSL)	4
2	Elements' Adaptation	5
2.1	Overview	5
2.2	ECS Modifications	6
2.3	Systems In Action	9
2.4	Event Manager	10
3	Renderer	12
3.1	Textures	12
3.2	Rendering Systems Architecture	13
3.3	Forward Rendering	15
3.4	Deferred Rendering	17
3.5	Custom Rendering Systems	19
4	Shaders	20
4.1	Shading Languages: WGSL and GLSL	20
4.2	WGSL Syntax Basics / Control flow	21
4.3	WebGPU forward / deferred rendering shaders	22
4.4	Limitations	24
5	Performance	25
6	Examples	28
6.1	Introduction Example	28
6.2	Textures Example	29
6.3	Lighting	30
6.4	Skybox	31
6.5	Normal Mapping	32
6.6	Parallax Occlusion Mapping	33
6.7	Shadows	34
6.8	Screen Space Ambient Occlusion	36
	References	37

List of Figures

1	Visual representations of the Entity with it's graphics related components.	2
2	Example of a scene rendered with the OpenGL renderer.	2
3	Difference between OpenGL and WebGPU. Image taken form WebGPU c++ guide[6].	3
4	Main rendering operation.	5
5	System implementation.	6
6	Difference between the forward and deferred rendering in use stages.	7
7	Mesh and Material components for WebGPU.	7
8	Simple example of components holding data and how they are being used inside a simple system.	8
9	Base system class implementation.	9
10	Example of adding a system in the scene.	9
11	GLFW vs SDL2.	10
12	visualization of Event system.	11
13	Representation of the enum holding the supported event types of the engine.	11
14	visualization of Textures and TextureLib.	12
15	Example of emplacing a texture in the <code>TextureLib</code>	12
16	Example of emplacing a skybox in the <code>TextureLib</code>	12
17	Example of emplacing a <code>rg32float</code> noise texture in the <code>TextureLib</code>	13
18	visualization of the Rendering.	13
19	Rendering system API.	14
20	visualization of the Rendering Subsystems.	14
21	Image produced from the Forward rendering sub-system.	15
22	Code representation of the <code>ForwardRenderPass</code> subsystem initialization.	16
23	Example of a draw implemition using the <code>WGPU</code> API.	16
24	Simple example using just the forward rendering pipeline.	16
25	Image produced from the Deferred rendering sub-system.	17
26	Intermediate textures stored in the g-buffer.	17
27	How the data in the G-buffer are being arranged.	18
28	Scene rendered without <code>SSAO</code> enabled vs with <code>SSAO</code> enabled.	19
29	Comparison of glsl (left) and wgsl (right).	20
30	Scalar types of WGSL.	21
31	Struct in WGSL.	21
32	Accepted vertex stages attributes.	22

33	Uniforms in WGSL.	22
34	Simple shader for forward rendering.	23
35	G-Buffer textures in use in the fragment shader.	24
36	Filling the depth buffer with the linearized depth stored in the normal texture.	24
37	Performance comparison between OpenGL renderer (left) and WGPU renderer (right) on a basic scene with just a skybox and a simple mesh. The performance was recorded with RivaTuner Statistics. .	25
38	Performance metrics recorded on a ryzen 7 processor, and an AMD 7900 xtx graphics card.	26
39	Example of the performance decrease due to the high usage of the rendering subsystems in a relatively complex scene with SSAO and other subsystems active.	27
40	Example of adding entities and components in the Scene.	28
41	Example of adding systems and filters in the Scene.	29
42	A simple example using the WebGPU renderer of Elements. . . .	29
43	Example of baking a texture for later usage.	29
44	Example of how to set the texture and the sampler for a shader component.	30
45	A scene with a mesh mapped with a texture.	30
46	Example of adding a light entity with the necessary components. .	30
47	Scene with a mesh using a Blinn-Phong shader and a light source.	31
48	Gathering the paths of the skybox textures and insert them to the skyboxComponent.	31
49	Example of a scene with a skybox.	32
50	Example of a normal texture.	32
51	Example of a Scene Pointing Out the Difference Between Normal Mapping and Simple Coloring.	33
52	Textures used in the Parallax Occlusion Mapping example.	33
53	Example of a scene with an entity with Parallax Occlusion Mapping.	34
54	Adding a camera component to the light entity.	35
55	Attaching a ShadowAffectedComponent to the correspondent af- fected entity.	35
56	Example of a scene with an entity with a shadow.	35
57	Setting the SSAO parameter to true in order to enable the SSAO rendering sub-system.	36
58	Figure capturing the difference between real time rendering with SSAO and without SSAO.	36

1 Introduction and Overview

1.1 The Elements framework

The Elements framework[7] stands out in the field of Computer Graphics (CG) by merging the Entity-Component-System (ECS) architecture with Scenegraphs. This innovative approach supports a broad range of applications, from basic Computer Graphics to advanced topics like Deep Learning (DL) for Scientific Visualization (SciViz), and even more specialized areas such as Geometric Deep Learning and Geometric Algebra. Written in Python, Elements is designed to be accessible and versatile, catering to both newcomers and seasoned researchers in the field.

Currently, Elements utilizes OpenGL[5] as its rendering backbone. While OpenGL has been a reliable technology, its capabilities are increasingly overshadowed by newer technologies that offer superior performance and modern features. In response to this need for modernization, this thesis focuses on integrating WebGPU into the Elements framework. Specifically, it involves implementing a new rendering engine interface.

The primary objective of this work is to enhance the Elements framework by replacing its existing OpenGL renderer with a WebGPU-based solution. This integration aims to leverage the advanced capabilities of WebGPU[6] to improve rendering performance and take advantage of contemporary graphics hardware. By doing so, this work seeks to address the limitations of the current OpenGL renderer and align the framework with modern graphics standards.

This sets the stage for a detailed exploration of the integration process, including the design, implementation, and evaluation of the new rendering engine. The following chapters will provide a thorough examination of existing technologies and frameworks related to graphics rendering, describe the methodology employed in this research, present the results of the integration, and discuss the implications and future directions.

1.2 Elements' OpenGL Rendering Overview

The Elements framework currently uses a simple forward rendering approach to generate images from 3D scenes. This technique is integrated with the Entity-Component-System (ECS) architecture, which helps manage shaders and scene objects effectively.

In this setup, each scene object is associated with a shader component containing the necessary shader data and programs. During the rendering process, the framework employs a Breadth-First Search (BFS) algorithm to traverse the

scene graph, efficiently processing objects in a hierarchical structure.

As the scene is traversed, the framework initializes the shader components and the render mesh components for each object (*Figure 1*). The render mesh component is crucial as it contains and organizes vertex stage data, including vertices, indices, normals, and texture coordinates (UVs). This data is essential for defining the shape and appearance of each object.

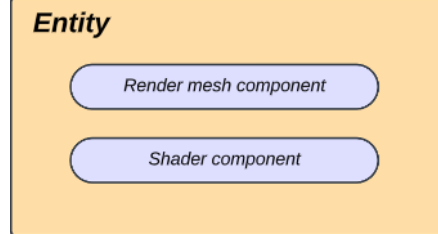


Figure 1: Visual representations of the Entity with its graphics related components.

Once the shader and render mesh components are initialized, they are sent to the GPU. In the forward rendering approach, the GPU uses this data to render each object in a single pass (*Figure 2*). The shader programs handle the lighting and shading calculations directly based on the vertex data provided.

After preparing and sending the rendering components to the GPU, the final step involves updating the uniform variables within the shader programs. This is done by setting values through shader-provided functions, ensuring that the shaders receive the necessary information for accurate rendering.

Overall, the simple forward rendering approach in Elements involves preparing and sending shader and render mesh components, including vertex stage data, to the GPU, and updating shader uniforms to achieve the final rendered output. This method effectively applies lighting and shading calculations in a single pass, handling the rendering process efficiently.

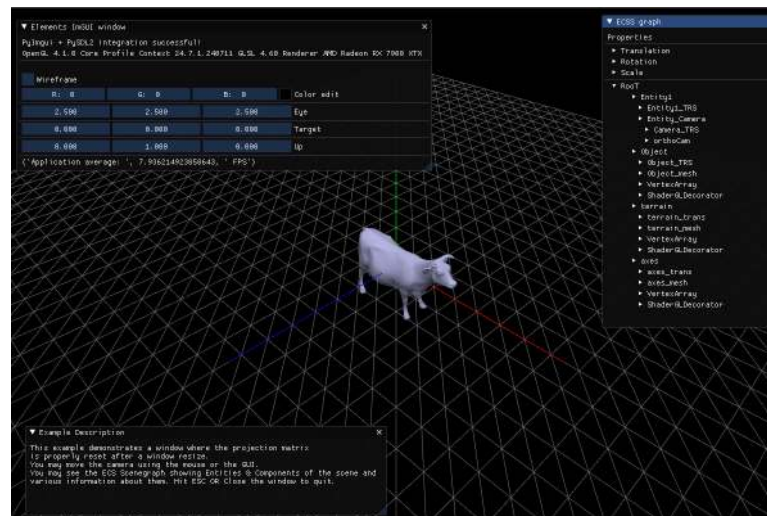


Figure 2: Example of a scene rendered with the OpenGL renderer.

1.3 What is WebGPU

WebGPU is a modern graphics API developed to offer high-performance access to GPU resources in web environments. It represents a significant advancement over older web graphics technologies like WebGL by providing more control and efficiency for rendering and computation tasks.

The primary motivation behind WebGPU is to meet the growing demands for advanced graphics and compute capabilities on the web. Unlike WebGL, which is based on the older OpenGL ES 2.0 API, WebGPU is designed to align with contemporary graphics APIs such as Vulkan[12], DirectX 12[4], and Metal[10]. This allows WebGPU to leverage the full potential of modern GPU architectures, offering developers a more powerful and flexible toolset.

One of the key features of WebGPU is its low-level access (*Figure 3*) to the GPU, which provides developers with detailed control over how rendering and compute operations are executed. This low-level interface is similar to the control available in native graphics APIs, enabling more precise optimization of performance and resource management. WebGPU supports advanced features like compute shaders, which facilitate general-purpose GPU computations beyond traditional graphics rendering. This capability allows for more complex and efficient processing of data on the GPU.

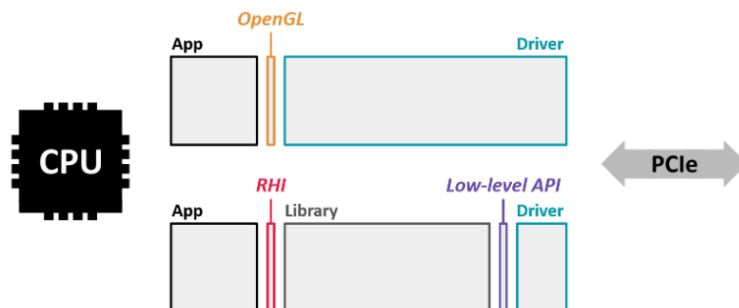


Figure 3: Difference between OpenGL and WebGPU. Image taken from WebGPU c++ guide[6].

Another significant improvement of WebGPU over WebGL is its enhanced resource management. WebGPU introduces explicit control over memory allocation and synchronization, helping to optimize performance and resource usage for high-demand applications. This is in contrast to WebGL’s more abstracted and less configurable approach.

WebGPU is designed to provide a consistent experience across different platforms and devices, adhering to web standards while offering powerful capabilities. It aims to deliver better performance and a broader set of features compared to WebGL and OpenGL, making it well-suited for applications that require high-performance graphics or complex computations, such as gaming, scientific visu-

alization, and simulations.

In summary, WebGPU represents a major leap forward in web graphics technology. It provides modern capabilities and performance improvements for creating sophisticated web applications by leveraging the latest advancements in GPU hardware. As support for WebGPU grows, it is expected to play a crucial role in advancing graphics and compute technologies.

1.4 Programming Languages (Python, WGSL)

Python is a high-level, interpreted programming language renowned for its simplicity and readability. Its clear syntax and extensive library ecosystem make it a preferred choice for a wide range of applications, from web development to scientific computing. Python is especially prominent in the field of artificial intelligence (AI) and machine learning (ML), where it is used extensively for developing algorithms, training models, and processing data. The language's rich collection of libraries and frameworks, such as TensorFlow, PyTorch, and scikit-learn, enables rapid development and deployment of AI solutions. Python's ease of use, combined with its powerful tools for data analysis and manipulation, positions it as a leading language for AI research and applications.

WGSL (WebGPU Shading Language)[1] is a specialized shading language designed for use with the WebGPU API. It is intended to define and control the behavior of shaders within the WebGPU framework, providing a modern and efficient approach to graphics and compute programming. WGSL allows developers to write shaders for both graphics rendering and general-purpose computations on the GPU. Its design focuses on being both expressive and user-friendly, catering to the advanced requirements of contemporary graphics programming and ensuring effective resource management and computation.

In summary, Python excels as a versatile language with a strong presence in AI and ML, thanks to its extensive libraries and ease of use. WGSL, on the other hand, offers a modern solution for shader programming within the WebGPU environment, enabling advanced graphics and compute capabilities.

2 Elements' Adaptation

2.1 Overview

The goal of the adaptation within Elements is to create a flexible rendering framework capable of handling both forward and deferred rendering techniques, or even a combination of both. This flexibility is essential for enabling a wide range of custom stages and special effects, allowing developers to tailor the rendering pipeline to their specific needs. The ultimate objective is to achieve seamless compatibility within the framework, making it possible to switch between or integrate these rendering methods as required.

On the surface, the renderer's primary responsibilities include preparing and resizing textures, producing the final rendered image, and efficiently blitting this result to the screen on a screen-sized quad (*Figure 4*). This design ensures that regardless of the underlying rendering technique used, the output remains consistent and visually accurate, providing a robust foundation for further customization and enhancement.



Figure 4: Main rendering operation.

To achieve this level of flexibility and compatibility within the Elements framework, several modifications were necessary on the CPU side, particularly within the Entity-Component System (ECS) that governs the renderer's operation. Since WebGPU operates at a lower level compared to OpenGL, additional information and processing need to be incorporated into the components and systems. This ensures that everything is properly prepared when it comes time to send data to the GPU.

Key adjustments include preparing the shaders with a custom parser, which allows for the extraction of essential data such as attributes and uniforms, functionality that is native in OpenGL but requires explicit handling in WebGPU. Additionally, textures must be carefully prepared, and the rendering pipeline needs to be set up with precise pipeline data for each draw call. These steps are crucial for meeting the specific requirements of the WebGPU API, which demands a more detailed and hands-on approach compared to OpenGL. These enhancements ensure that the framework can maintain the same functionality and performance, despite the differences in the underlying graphics API.

2.2 ECS Modifications

The systems used for the WebGPU implementation are designed with simplicity to ensure the code runs efficiently. Each specialized system derives from a super-class (*Figure 5*), which provides a function that filters the relevant components upon creation and modifies them during the update phase. This approach ensures speed and stability when modifying the newly added components, some of which are among the most important ones mentioned below.

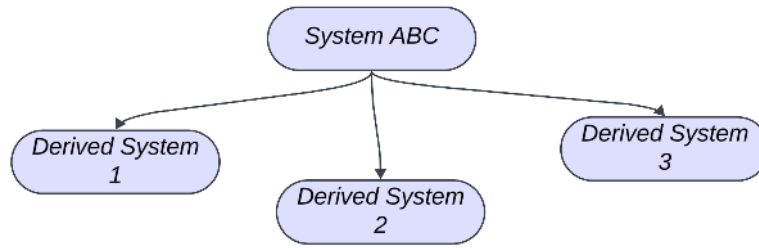


Figure 5: System implementation.

To maintain the same functionality within the Elements framework while adapting to WebGPU, several changes and additions were necessary within the Entity-Component System (ECS). On the component side, the primary focus was on adapting the shader component to accommodate the data required by the GPU, such as uniform variables and attribute slots. This adaptation was crucial to ensure compatibility with WebGPU’s lower-level operations.

Given the goal of supporting both forward and deferred rendering techniques (*Figure 6*), the shader component needed to be flexible enough to handle the requirements of both approaches. For the forward rendering shader component, the primary focus was on managing the vertex and fragment shaders, which reflect data from the WGSL (WebGPU Shading Language). These shaders are responsible for processing the attributes and uniforms specific to each entity.

For the deferred rendering approach, the shader component differs slightly. In this case, the geometry remains consistent across entities, meaning that the vertex shader is the same for all entities containing a deferred shader component. Therefore, the deferred shader component primarily focuses on the fragment shader, which handles the essential data required for rendering. The final lighting calculations, which would typically involve a light stage shader, are deferred to screen space and applied after the geometry pass. This separation allows for more efficient processing and the flexibility to apply complex lighting effects in a post-processing step.

To support these components, we developed the corresponding rendering sys-

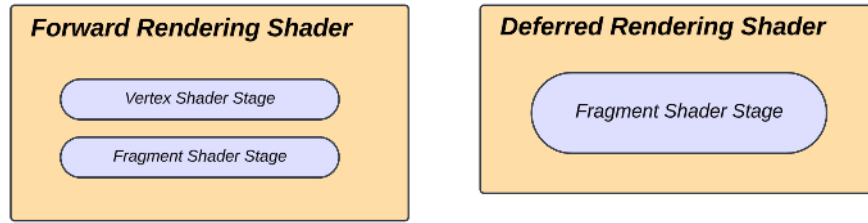


Figure 6: Difference between the forward and deferred rendering in use stages.

tems that interact with these components. These systems are responsible for performing reflection on the shader data, preparing the shaders according to their type—whether forward or deferred—and ensuring they are correctly configured for WebGPU. This ensures that each shader component is accurately set up, reflecting the necessary data and ready to be used for rendering, depending on the chosen rendering method.

Furthermore, the mesh and material components (*Figure 7*) play crucial roles in the rendering process. The mesh component is responsible for holding the data necessary for the shader attributes, including vertices, indices, UVs, tangents, bitangents, and colors specific to each mesh. Importantly, it is not necessary to provide all of this data; the mesh system will convert and upload only the attributes that are provided. The mesh system then transforms the available data into GPU-compatible formats and uploads it to the appropriate buffers, effectively storing the GPU buffer pointers for later use.

The material component manages data related to pipeline specifications, including settings for culling, primitive type, depth-stencil configurations, and color blending (with color blend settings being relevant primarily in forward rendering). This organization ensures that each mesh and material can be efficiently processed and rendered according to the specific requirements of the WebGPU environment.

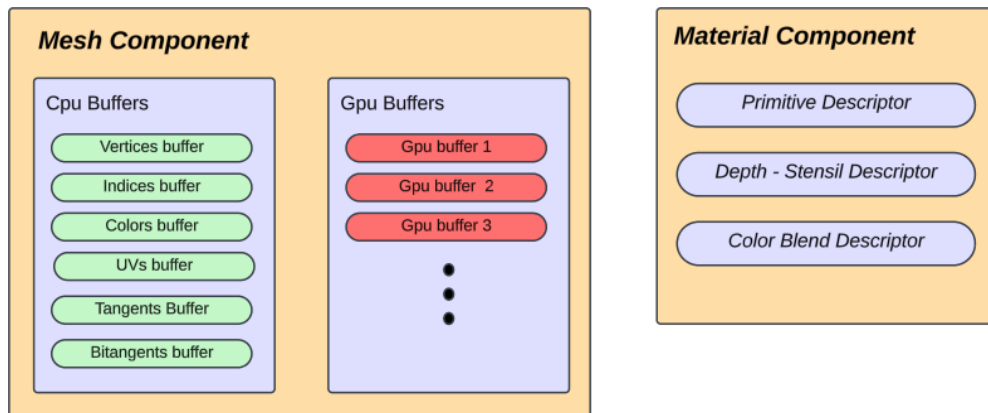


Figure 7: Mesh and Material components for WebGPU.

Some components of lesser importance include the light component, which

holds data for the light entities in the scene; the light affection component, which references the light entity affecting a particular mesh; and the shadow affection component, which marks entities that will participate in the shadow casting stage and utilize the shadow depth map produced.

The modifications to the Entity-Component System (ECS) were crucial for adapting the Elements framework to WebGPU. Key updates included refining the mesh component to handle various shader attributes and ensuring that only provided data is processed, as well as enhancing the material component to manage pipeline specifications effectively. Additionally, supporting components such as light, light affection, and shadow affection were integrated to address lighting and shadow casting requirements.

Importantly, the ECS framework remains flexible and extensible, allowing users to create and integrate their own custom components and systems (*Figure 8*). Users can define custom components and develop systems that modify these components as needed, ensuring that they can tailor the framework to meet specific needs and applications while maintaining efficiency and performance.

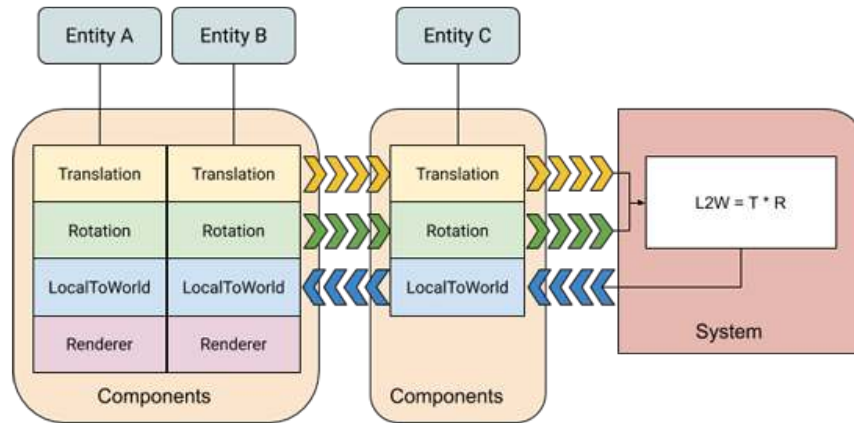


Figure 8: Simple example of components holding data and how they are being used inside a simple system.

2.3 Systems In Action

To create and run a system within a scene, you need to extend the system super-class and implement the `on_create` and `on_update` functions (*Figure 9*). The `on_create` function is used to initialize the system and set up any necessary component filters or other configurations when the system is first created. The `on_update` function is responsible for modifying the components during the main update loop, allowing the system to process and update component data as needed.

```

1  class System(object):
2      """
3      Base class for all systems in the ECS framework.
4
5      Systems are responsible for processing entities that match specific component filters.
6      """
7
8      def on_create(self, entity: Entity, components: Component | list[Component]):
9          """
10         Method called when a new entity is created and matches the system's filters.
11
12         This method should be overridden by subclasses to define specific behavior.
13
14         :param entity: The entity being created.
15         :param components: The components of the entity.
16         """
17
18         pass;
19
20     def on_update(self, ts, entity: Entity, components: Component | list[Component], event):
21         """
22         Method called during the update phase for each entity that matches the system's filters.
23
24         This method should be overridden by subclasses to define specific behavior.
25
26         :param ts: Timestamp of the update.
27         :param entity: The entity being updated.
28         :param components: The components of the entity.
29         :param event: The event to handle during the update.
30         """
31
32         pass;
33 
```

Figure 9: Base system class implementation.

Now, you simply need to add the system to the scene along with specifying the component types it will act upon (*Figure 10*).

```

1  Scene().add_system(MySystem([component_type_1, component_type_2, ...]))

```

Figure 10: Example of adding a system in the scene.

All of the components must be added to the scene First. If that's not the case the On_Create function of the systems will not work properly.

2.4 Event Manager

Originally, the Elements framework used PySDL[13] to handle OpenGL and manage events such as input and window events. However, this approach was no longer viable because SDL does not officially support WebGPU. As a result, there is no elegant way to present a WebGPU image context within the SDL framework. Therefore, we had to switch libraries and move to GLFW[3] (*Figure 11*). GLFW is a more lightweight library that doesn't require .dll files and, while it may not offer as much functionality as SDL, it is faster and provides a window through the window manager without binding it to a specific API like OpenGL. This means we can instantiate a GLFW window without initializing an API, and then initialize WebGPU separately.

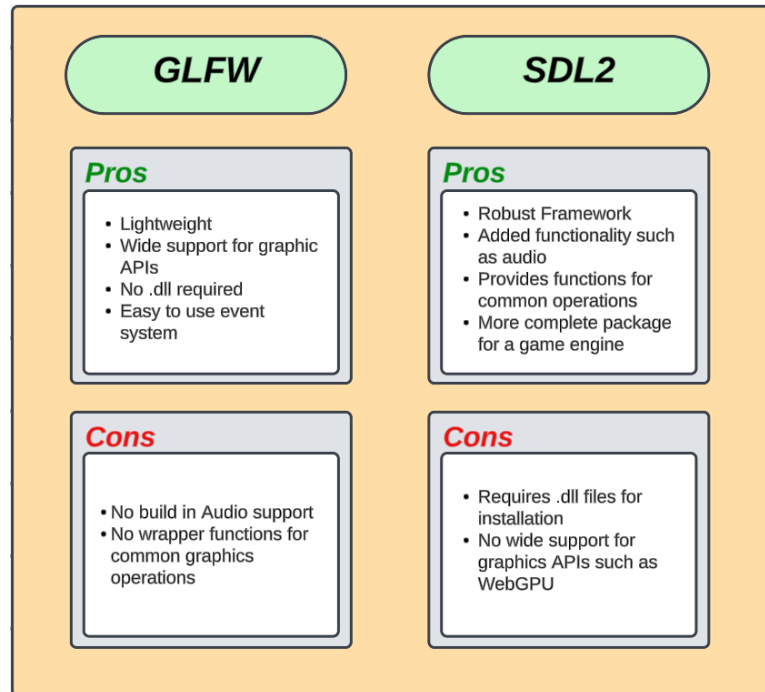


Figure 11: GLFW vs SDL2.

After setting up GLFW, our next focus was on the event system, as GLFW handles events quite differently from SDL2. To manage this, we've implemented an event bus to which every GLFW callback reports. We then used a function called `poll_events` to retrieve the latest events recorded in the event bus (*Figure 12*). Essentially, for each important event, we created a GLFW callback.

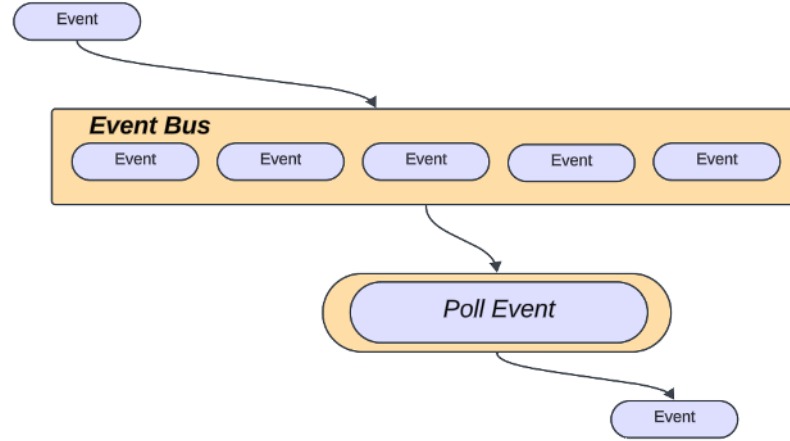


Figure 12: visualization of Event system.

At the end of each callback function, we register the event with the appropriate event type into the event bus. Finally, with the implemented window `poll_events` function, we retrieve the latest event and feed it into the systems' update function for processing (*Figure 13*).

```

class EventTypes(Enum):
    KEY_PRESS = 1
    KEY_RELEASE = 2
    KEY_REPEAT = 3
    MOUSE_BUTTON_PRESS = 4
    MOUSE_BUTTON_RELEASE = 5
    MOUSE_MOTION = 6
    CURSOR_POS = 7
    SCROLL = 8
    WINDOW_SIZE = 9
    WINDOW_CLOSE = 10
    WINDOW_REFRESH = 11
    WINDOW_FOCUS = 12
    FRAMEBUFFER_SIZE = 13
    STICK = 14
    POINTER_ENTER = 15
    POINTER_LEAVE = 16
    FOCUS = 17

```

Figure 13: Representation of the enum holding the supported event types of the engine.

3 Renderer

3.1 Textures

For the implementation, we needed a method to create and store textures that could be accessed from anywhere within the framework. This is crucial because we not only want to access user-generated textures but also need to store textures used by various subsystems during the rendering stage. To achieve this, we designed a texture prototype struct that holds the texture's bytes, its size, the GPU data, and the GPU view associated with that texture (*Figure 14*). We then created a singleton called `TextureLib`, which stores every texture produced both inside and outside the core, with each texture assigned with simple name for easy access.

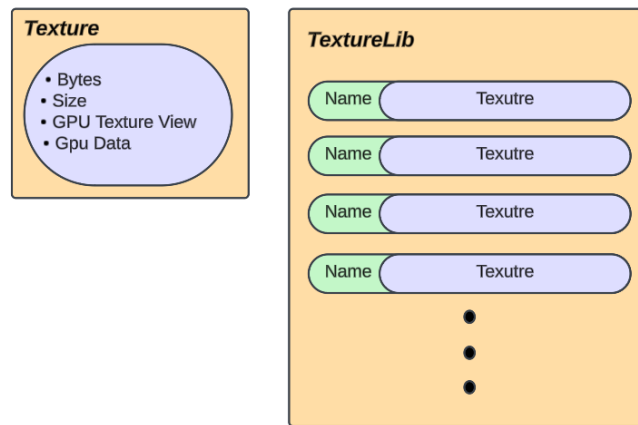


Figure 14: visualization of Textures and TextureLib.

With this `TextureLib` in place, we can implement functions to create various types of textures and bake them into the GPU. First and foremost, we can create a simple texture by calling the `make-texture` function (*Figure 15*), which generates a basic `rgb8unorm` texture and stores it with the appropriate name.

```
43 TextureLib().make_texture(name="plane_texture", path=definitions.TEXTURE_DIR / "dark_wood_texture.jpg")
```

Figure 15: Example of emplacing a texture in the `TextureLib`.

Having created that, we implemented a function to store skyboxes (*Figure 16*). Skyboxes are simply six textures combined and fed into the GPU as a single texture with multiple layers. The implementation is quite similar to that of a simple texture and is actually very straightforward to execute.

```
3 TextureLib().make_skybox(name, skybox_texture_paths)
```

Figure 16: Example of emplacing a skybox in the `TextureLib`.

Finally, we implemented a function that generates a texture to be used as noise (*Figure 17*). This is achieved by applying random rotations and taking the size and array depth as arguments, resulting in an `rg32float` texture that can be used for noise.

```

155 | TextureLib().create_noise_texture(name="ssao_noise_gfx", config={
156 |     "width": SSAO_NOISE_TEXTURE_SIZE,
157 |     "height": SSAO_NOISE_TEXTURE_SIZE,
158 |     "depth_or_array_layers": 1
159 | })

```

Figure 17: Example of emplacing a `rg32float` noise texture in the `TextureLib`

To sum up, textures in our framework are managed through a centralized `TextureLib`, which facilitates the creation, storage, and access of textures from anywhere within the framework. We implemented a texture prototype struct to hold essential data, including texture bytes, size, GPU data, and a GPU view. This setup allows for the creation of various textures, such as simple `rgb8unorm` textures, skyboxes composed of multiple layers, and `rg32float` textures for noise generation. Each texture is stored in `TextureLib` with a unique name for easy reference during rendering.

3.2 Rendering Systems Architecture

Now that we have a mechanism for storing textures, the next step is implementing the rendering system. Previously, the OpenGL renderer simply took an entity, retrieved the render mesh component and shader component, and rendered it in the scene (*Figure 18*).

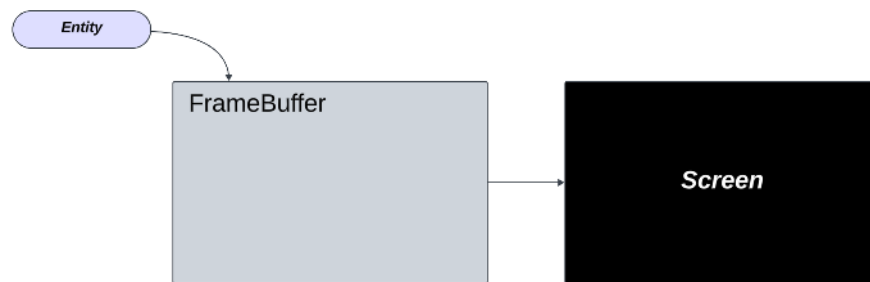


Figure 18: visualization of the Rendering.

The WebGPU renderer is implemented in a manner consistent with our ECS (Entity-Component-System) approach (*Figure 19*). We have a base class that contains filtering functions for the rendering subsystems and operates on the same components present in the scene. The derived render system must implement the `on-create`, `on-prepare`, and `on-render` functions. Subsequently, in the main rendering function, we determine when to invoke these render systems to generate and store intermediate textures.

```

6 class RenderSystem(object):
7     """
8     Base class for rendering systems. Handles filtering of entities based on required components
9     and provides hooks for creating, preparing, and rendering entities.
10    """
11
12    def __init__(self, filters: list[type]):
13        self.filters = filters
14
15    def on_create(self, entity: Entity, components: Component | List[Component]):
16        """
17        Hook method called when an entity is created.
18
19        :param entity: The entity being created.
20        :param components: The components of the entity.
21        """
22        pass;
23
24    def on_prepare(self, entity: Entity, components: Component | List[Component], command_encoder: wgpu.GPUCommandEncoder):
25        """
26        Hook method called when an entity is prepared for rendering.
27
28        :param entity: The entity being prepared.
29        :param components: The components of the entity.
30        :param command_encoder: Command encoder for GPU commands.
31        """
32        pass;
33
34    def on_render(self, entity: Entity, components: Component | List[Component], render_pass: wgpu.GPURenderPassEncoder | wgpu.GPUComputePassEncoder):
35        """
36        Hook method called when an entity is rendered.
37
38        :param entity: The entity being rendered.
39        :param components: The components of the entity.
40        :param render_pass: Render pass encoder for GPU rendering.
41        """
42        pass;
43
44    def on_post_render(self):
45        pass;

```

Figure 19: Rendering system API.

This approach ensures that rendering systems are invoked in the correct order, and actual computation occurs only if there is an entity with the required components as specified by the render subsystem filters. This design provides a stable swapchain that creates textures in the correct sequence for the production of the final image (*Figure 20*). Additionally, it allows for dynamic control based solely on the existing components in the scene.

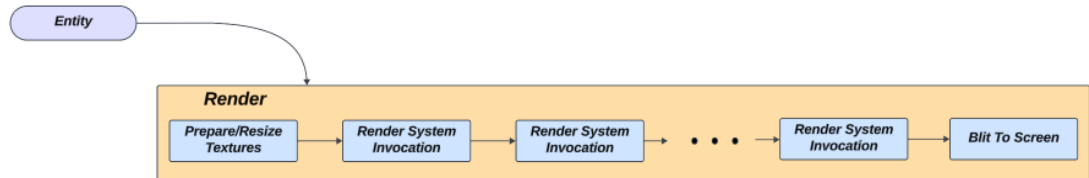


Figure 20: visualization of the Rendering Subsystems.

What can a rendering subsystem do? Essentially, anything. For each rendering system invocation, we create a render pass with its descriptor and pass it to the subsystem. With this, the subsystem can perform a wide range of tasks—from using a static shader and executing specific actions to writing data to the GPU, reading data from the GPU, making draw calls, or dispatching compute shaders. In short, the rendering subsystem has the flexibility to handle any GPU-related operation.

3.3 Forward Rendering

The forward rendering[11] subsystem is one of the core rendering techniques implemented within our framework, designed to handle the direct rendering of entities in a scene. In this approach, all visible objects are rendered in a single pass, where each entity’s mesh, material, and associated shaders are processed together. The subsystem operates by using component type filters to identify and act on all components of the specified types within the scene. It then collects the necessary data from these mesh and material components and applies the relevant shaders to produce the final image (*Figure 21*).

In this method, lighting calculations are performed during the rendering of each object, meaning that the number of lights and the complexity of the shaders directly impact rendering performance. Despite these challenges, forward rendering remains a widely used technique due to its straightforward implementation and compatibility with a broad range of hardware.

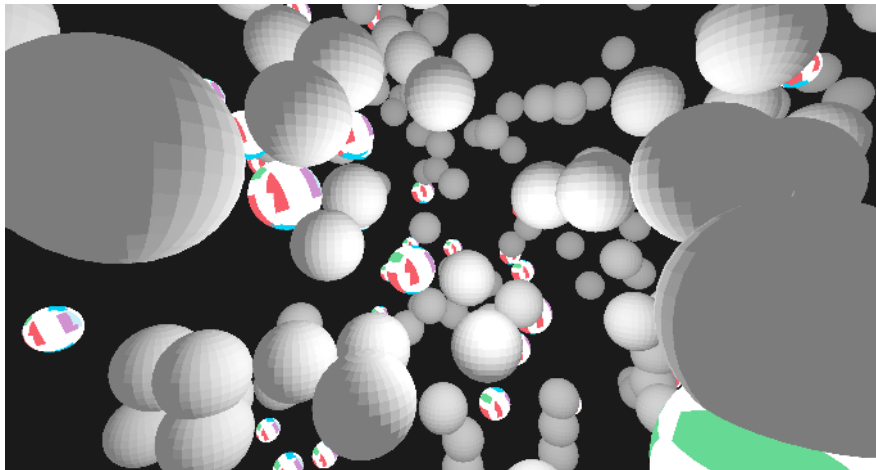


Figure 21: Image produced from the Forward rendering sub-system.

To implement the forward rendering subsystem within the architecture mentioned above, the first step is to determine the types of components this subsystem will act upon (*Figure 22*). The obvious choice is the material component, which provides essential information for configuring the rendering pipeline, including depth buffer settings, primitive types, and color blending options. Next, the mesh component is required to supply the vertex stage attributes, such as vertices, indices, and other geometric data, which need to be sent to the GPU. Finally, the subsystem relies on the `ForwardRenderingShader` component, which holds the shader contexts for both the vertex and fragment stage shaders, ensuring that the appropriate shaders are applied during the rendering process.

In the `on_prepare` function, the next step is to create the descriptor for the bind groups and map these bind groups accordingly. This process involves

```
ForwardRenderPass([MeshComponent, MaterialComponent, ForwardShaderComponent]))
```

Figure 22: Code representation of the `ForwardRenderPass` subsystem initialization.

specifying which data corresponds to which bindings, ensuring that the GPU correctly associates the data with the appropriate resources in the shaders. Once the bind groups are set up, the next task is to create the rendering pipeline. With the bind groups ready, we can configure the pipeline, which serves as the descriptor for the upcoming draw calls that will be executed in the on-render function.

In the `on-render` function, the primary task is to issue the draw call and send the correct data to the GPU (*Figure 23*). This is done using the descriptors we previously set up, such as the bind groups and the pipeline descriptor. These elements ensure that the GPU has all the necessary information to render the scene correctly.

```
158 |         render_pass.set_pipeline(material.pipeline)
159 |         render_pass.set_index_buffer(mesh.buffer_map[MeshComponent.Buffers.INDEX.value], wgpu.IndexFormat.uint32)
160 |         for name, attr in shader.attributes.items():
161 |             render_pass.set_vertex_buffer(slot=attr['slot'], buffer=mesh.buffer_map[name])
162 |         for bind_group_id, bind_group in enumerate(shader.bind_groups):
163 |             render_pass.set_bind_group(bind_group_id, bind_group, [], 0, 99)
164 |         render_pass.draw_indexed(mesh.indices_num, 1, 0, 0, 0)
```

Figure 23: Example of a draw implementation using the WGPU API.

To sum up, the forward rendering subsystem efficiently handles the rendering of scene entities by processing essential components—material, mesh, and forward rendering shaders. The `on-prepare` function configures bind groups and the rendering pipeline to ensure proper data mapping for shaders. Subsequently, the `on-render` function issues draw calls using these configurations, rendering each entity with the appropriate settings. This subsystem plays a vital role in generating the initial visual output (*Figure 24*), setting the stage for subsequent rendering processes.

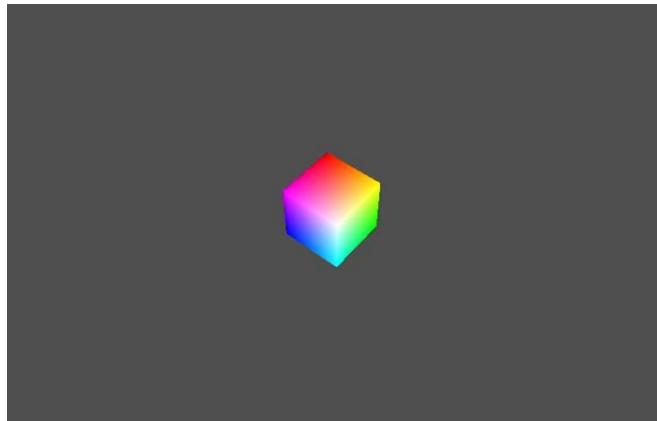


Figure 24: Simple example using just the forward rendering pipeline.

3.4 Deferred Rendering

The deferred rendering[9] subsystem is a sophisticated rendering technique designed to handle complex scenes with multiple light sources more efficiently than forward rendering. Unlike forward rendering, which processes lighting calculations during the initial rendering pass, deferred rendering separates the rendering process into distinct stages. This method first captures the scene’s geometry and attributes into multiple intermediate textures, known as the G-buffer. Subsequently, lighting and shading calculations are performed in a separate pass, using the information stored in the G-buffer (*Figure 25*).



Figure 25: Image produced from the Deferred rendering sub-system.

To achieve this outcome, we need to implement two distinct rendering subsystems. The first subsystem handles the geometry pass, filling the G-buffer with essential data (*Figure 26*), while the second subsystem manages the lighting calculations in screen space. During the geometry pass, the G-buffer is populated with various textures, including the albedo texture, normal texture, and position texture. All of these textures are stored in the `TextureLib` we established earlier, ensuring they are readily accessible for the subsequent lighting pass.

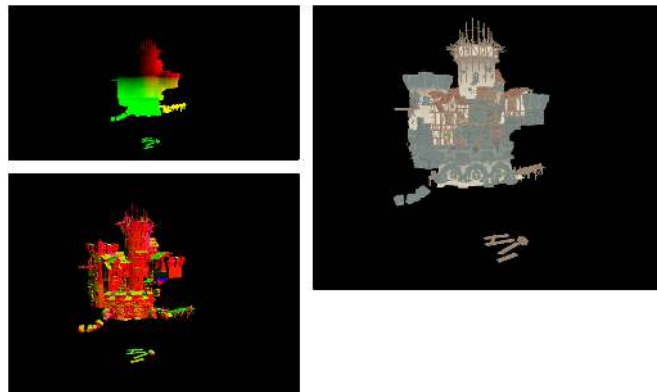


Figure 26: Intermediate textures stored in the g-buffer.

The arrangement of data within the textures of the G-buffer is carefully structured to optimize the deferred rendering process. In the position texture, we store the x , y , and z values of the fragment's position in world space. The albedo texture holds the r , g , and b color values of each pixel. For the normal texture, we store the x , y , and z components of the fragment's normal. Additionally, the w channel of the normal texture is used to store the linearized depth value relative to the camera position (*Figure 27*).

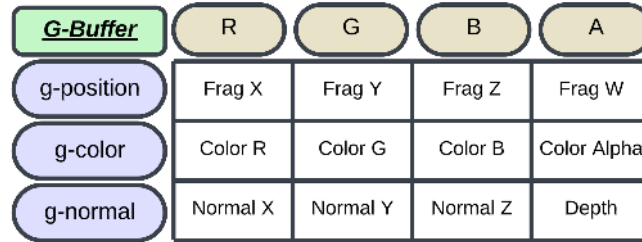


Figure 27: How the data in the G-buffer are being arranged.

Following the geometry pass, the lighting stage sub-system operates similarly to the forward rendering sub-system, but with a key distinction: only the fragment shader is involved. Since all necessary data is already stored in the G-buffer, the lighting calculations occur entirely in screen space within the fragment shader. This approach is a more optimized method for rendering multiple lights in a scene, allowing for complex lighting effects without sacrificing performance.

The deferred rendering subsystem offers a powerful and efficient approach to handling complex lighting in scenes with multiple light sources. By separating the geometry and lighting stages, and utilizing the G-buffer to store key data, this method allows for advanced lighting calculations to be performed in screen space. This not only optimizes performance but also enhances the flexibility of rendering, making it an ideal solution for scenes requiring intricate visual effects without compromising on speed or resource efficiency.

3.5 Custom Rendering Systems

With the flexible rendering architecture provided, it's possible to create custom subsystems tailored to specific needs, allowing for the baking of any texture and storing it in the `TextureLib`. For instance, subsystems can be designed to leverage deferred rendering for advanced effects such as **SSAO** (Screen Space Ambient Occlusion)[2]. Implementing **SSAO** typically requires two subsystems: one to perform the actual **SSAO** pass and another to blur the **SSAO** output (*Figure 28*). The result can then be integrated into the deferred lighting pass for enhanced lighting calculations.

Additionally, custom subsystems can be created for post-processing effects like anti-aliasing. An example of this would be implementing **FXAA** (Fast Approximate Anti-Aliasing)[14], which processes the image produced by the lighting pass to generate an anti-aliased output. This architecture not only supports complex effects but also empowers developers to innovate and customize their rendering pipeline as needed.

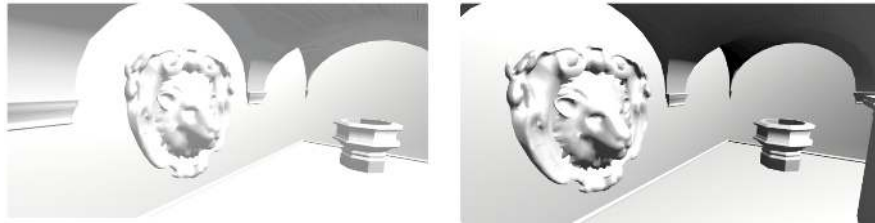


Figure 28: Scene rendered without **SSAO** enabled vs with **SSAO** enabled.

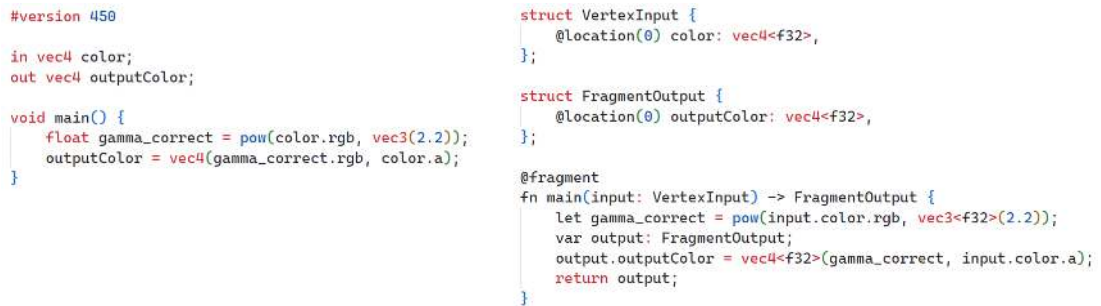
The provided rendering architecture offers a robust foundation for creating custom subsystems, enabling the development of specialized effects and post-processing techniques. By allowing developers to bake and store textures within the `TextureLib`, the system supports advanced features like **SSAO** and **FXAA**, while also offering the flexibility to integrate new effects seamlessly into the rendering pipeline. This adaptability ensures that the framework can be tailored to meet diverse rendering needs, fostering innovation and customization in visual effects.

4 Shaders

4.1 Shading Languages: WGSL and GLSL

As Elements expands to support both OpenGL and WebGPU rendering interfaces, it’s important to highlight the differences between their respective shading languages, GLSL and WGSL (*Figure 29*). GLSL has been the standard within Elements for shader development under OpenGL, providing a familiar environment for writing vertex and fragment shaders. With the introduction of WebGPU and its native shading language, WGSL, we now have the opportunity to leverage a language specifically designed to take full advantage of modern GPU architectures. While both GLSL and WGSL serve the same fundamental purpose—defining the GPU’s rendering processes—WGSL introduces new syntax, semantics, and features that better align with WebGPU’s structure. This addition enriches the Elements framework, allowing for more versatile and optimized rendering options.

WGSL is a high-level language, meaning it provides a more abstract and human-readable way to define shader programs. However, it is not directly understood by the GPU. Before being executed, WGSL code must be compiled down to machine instructions, which are the low-level commands the GPU can natively process and execute.



```
#version 450

in vec4 color;
out vec4 outputColor;

void main() {
    float gamma_correct = pow(color.rgb, vec3(2.2));
    outputColor = vec4(gamma_correct.rgb, color.a);
}
```

```
struct VertexInput {
    @location(0) color: vec4<f32>,
};

struct FragmentOutput {
    @location(0) outputColor: vec4<f32>,
};

@fragment
fn main(input: VertexInput) -> FragmentOutput {
    let gamma_correct = pow(input.color.rgb, vec3<f32>(2.2));
    var output: FragmentOutput;
    output.outputColor = vec4<f32>(gamma_correct, input.color.a);
    return output;
}
```

Figure 29: Comparison of glsl (left) and wgsl (right).

Overall, the syntaxes of GLSL and WGSL are quite similar, as both are shading languages designed to communicate with GPUs in a similar fashion. However, WGSL introduces some differences, particularly when compared to the Web-specific flavor of GLSL. These differences reflect WGSL’s design to align with the architecture and capabilities of WebGPU, making it a more optimized and modern choice for rendering in this context. Despite these changes, transitioning from GLSL to WGSL within Elements should feel familiar to those already accustomed to GPU programming.

4.2 WGSL Syntax Basics / Control flow

In WGSL, the syntax reflects a blend of influences, notably from Rust, which introduces a more explicit and structured approach to defining types compared to GLSL (*Figure 30*). While boolean values (`bool`) behave the same as in GLSL, scalar types in WGSL explicitly specify their bit width: `i32`, `u32`, and `f32` for 32-bit integers, unsigned integers, and floating-point numbers, respectively.

WGSL Scalar Types	
	<code>bool</code>
	<code>i32</code>
	<code>u32</code>
	<code>f32</code>

Figure 30: Scalar types of WGSL.

Vectors and matrices in WGSL require even more specificity. A vector of four integers, for instance, is written as `vec4<i32>`, making the component type explicit within angle brackets. This contrasts with GLSL’s more straightforward `vec4` syntax, where the type was implied.

Matrix types in WGSL are also detailed, using a verbose format like `mat4x4<f32>`, indicating a 4x4 matrix of floats, with no shorthand available. Constructing matrices is more cumbersome in WGSL compared to GLSL, where you could easily generate an identity matrix with `mat4(1.0)`. Currently, WGSL requires passing vectors to build matrices, but future updates may simplify this process.

```
struct Uniforms {
    proj: mat4x4f,
    view: mat4x4f,
    light_view: mat4x4f,
    light_proj: mat4x4f
};
```

Figure 31: Struct in WGSL.

Arrays follow a similar pattern of explicit typing, with the format `array<E,N>` where `E` is the element type, and `N` is the number of elements (*Figure 31*). For example, an array of eight 2D float vectors would be written as `array<vec2<f32>, 8>`. This explicitness, while initially verbose, provides clarity and precision in the shader code.

4.3 WebGPU forward / deferred rendering shaders

In forward rendering, the shader used in WebGPU closely resembles the one used in OpenGL, with the main differences arising from the syntax and structure variations between WGSL and GLSL. Given these differences, the forward rendering stage in WebGPU requires a shader file that includes both the vertex and fragment shaders.

For this iteration of the renderer, the vertex shader attributes follow a fixed naming scheme (*Figure 32*). The attributes accepted in the vertex shader are as follows:

Vertex Attributes
a_vertices
a_indices
a_uvs
a_normals
a_tangents
a_bitangents

Figure 32: Accepted vertex stages attributes.

Next, we need to define uniform buffers (*Figure 33*), and, if necessary, storage buffers. In this iteration, Element’s WebGPU renderer supports only read-only storage buffers, which are defined within the shader.

```
struct Uniforms {
    projectionMatrix: mat4x4f,
    viewMatrix: mat4x4f,
    modelMatrix: mat4x4f,
    color: vec4f,
    time: f32,
};

@group(0) @binding(0) var<uniform> u_Uniforms: Uniforms;
```

Figure 33: Uniforms in WGSL.

With all the data declared, the next step is to write the vertex and fragment shaders. The specific computations within these shaders depend on the user's requirements (*Figure 34*). Each shader must be marked with `@vertex` for the vertex shader and `@fragment` for the fragment shader to designate their entry points.

```

struct uniforms {
    projection: mat4x4f,
    view: mat4x4f,
    model: mat4x4f,
    color: vec3f,
    near_far: vec2f
};

@group(0) @binding(0) var<uniform> ubuffer: uniforms;

struct VertexInput {
    @location(0) a_vertices: vec3f,
}
struct VertexOutput {
    @builtin(position) Position: vec4<f32>,
};
struct FragOutput {
    @location(0) color: vec4f
};

@vertex
fn vs_main(
    in: VertexInput
) -> VertexOutput {

    var model = transpose(ubuffer.model);
    var view = transpose(ubuffer.view);
    var projection = transpose(ubuffer.projection);

    var out: VertexOutput;
    out.Position = projection * view * model * vec4<f32>(in.a_vertices, 1.0);
    return out;
}

@fragment
fn fs_main(
    in: VertexOutput
) -> FragOutput {
    var out: FragOutput;

    out.color = vec4f(ubuffer.color, 1.0);
    return out;
}

```

Figure 34: Simple shader for forward rendering.

Deferred rendering does not require a vertex stage shader, as the geometry is handled internally by the renderer and is consistent across all meshes provided by the user. Therefore, only the fragment shader is necessary. The approach to writing the fragment shader is similar to that used in forward rendering, with the key difference being the utilization of G-buffer textures (*Figure 35*). These textures provide specific information for each fragment, enabling more complex lighting calculations.

Lastly, since forward and deferred rendering can coexist within the same framebuffer, and because deferred rendering occurs first in screen space, we need

```

@group(0) @binding(0) var<uniform> ubuffer:    uniforms;
@group(1) @binding(0) var g_position_texture: texture_2d<f32>;
@group(1) @binding(1) var g_normal_texture:   texture_2d<f32>;
@group(1) @binding(2) var g_color_texture:    texture_2d<f32>;

```

Figure 35: G-Buffer textures in use in the fragment shader.

to ensure that the depth information calculated during the geometry pass is correctly written to the depth buffer (*Figure 36*). This step is crucial for preparing the framebuffer so that the forward rendering stage can blend both rendering methods together seamlessly.

```
out.depth = textureLoad(g_normal_texture, pos_texel, 0).a;
```

Figure 36: Filling the depth buffer with the linearized depth stored in the normal texture.

4.4 Limitations

While WGSL offers a range of features for shader programming, it is not without its limitations. Understanding these constraints is essential for effective shader development and ensuring compatibility with existing workflows. Below, we explore some of the notable limitations of WGSL.

WGSL currently lacks support for preprocessor directives such as `#define`, `#ifdef`, or `#ifdefined`. This design choice assumes that preprocessing will be managed on the client side, possibly through string replacement methods or template strings in JavaScript. Consequently, shaders written in WGSL may be more specialized and streamlined, potentially enhancing their readability once pre-processing is applied. However, it is worth noting that the absence of preprocessor support is a limitation of the initial WGSL specification, and this could change with future updates.

In WGSL, all data types are 32-bit, limiting you to `f32` (floating-point), `i32` (signed integer), and `u32` (unsigned integer). The language supports basic arithmetic and assignment operators but lacks certain features such as increment (`++`), decrement (`--`), and exponentiation (`**`). Additionally, while l-value swizzling is available, the absence of common assignment operators `+=`, `-=`, `*=`, `/=` requires more manual management of values during shader programming.

WGSL has limited support for operator overloading and built-in functions. For instance, while you can use the addition operator (`+`) to add a `float` to a `vec3` and have the float added to each component of the vector, the less-than operator (`<`) only works with vectors of the same dimension. This means you cannot directly compare a `vec3` with a `float`; instead, you must convert the `float` to a `vec3` before comparison, like `someVec3 < vec3<f32>(someFloat)`.

Additionally, many built-in functions, such as `pow(a, b)` and `clamp(x, a, b)`, require that all parameters be of the same vector type. For example, to use `clamp`, you would write `clamp(someVec2, vec2<f32>(0.0), vec2<f32>(1.0))` to ensure type consistency.

5 Performance

In this section, we focus on the performance gains achieved by using WebGPU over OpenGL, especially when comparing the two renderers in the context of this project.

Although the WebGPU renderer introduces additional complexity by employing a subsystem-based architecture, which increases CPU dependence, it still shows significant performance improvements (e.g. in *Figure 37* we conducted a test measuring the performance metrics of the two renderers on a basic scene containing a simple mesh and a skybox. As we can see despite the higher CPU usage of the WebGPU renderer due to its architecture the GPU usage was actually lower on the WebGPU and the frames per second increased from 207 to 526). This is due to the more advanced and robust graphics APIs offered by WebGPU. In contrast, the OpenGL renderer is a strictly forward rendering system, which can often become a bottleneck, particularly when handling complex scenes with multiple lights or post-processing effects. WebGPU, with its deferred and forward rendering capabilities, offers better handling of such workloads, enabling efficient parallelization and more optimized use of the GPU.



Figure 37: Performance comparison between OpenGL renderer (left) and WGPU renderer (right) on a basic scene with just a skybox and a simple mesh. The performance was recorded with RivaTuner Statistics.

To evaluate the performance differences between the WebGPU and OpenGL implementations, a series of tests were conducted. These tests involved rendering a scene with a skybox and a variable number of cubes. The cube count varied in increments, starting from 10 cubes, then increasing to 25 cubes, and finally 50 cubes. For each test case, the cubes were randomly translated within the scene, with their positions ranging from 10 to 200 units on each axis.

The performance metrics, most importantly frames per second, were recorded for both the WebGPU and OpenGL versions of the renderer (*Figure 38*). The aim was to assess how each rendering API handled an increasing number of objects in the scene, as well as the overhead introduced by additional draw calls and translation operations.

The test scene consisted of the following:

- A skybox rendering at all times.
- A variable number of cubes (10, 25, and 50), each randomly placed in a position within a range of 10 to 200 units on the X, Y, and Z axes.
- The cubes were assigned simple materials and for every frame their position was changing based on the range mentioned above.

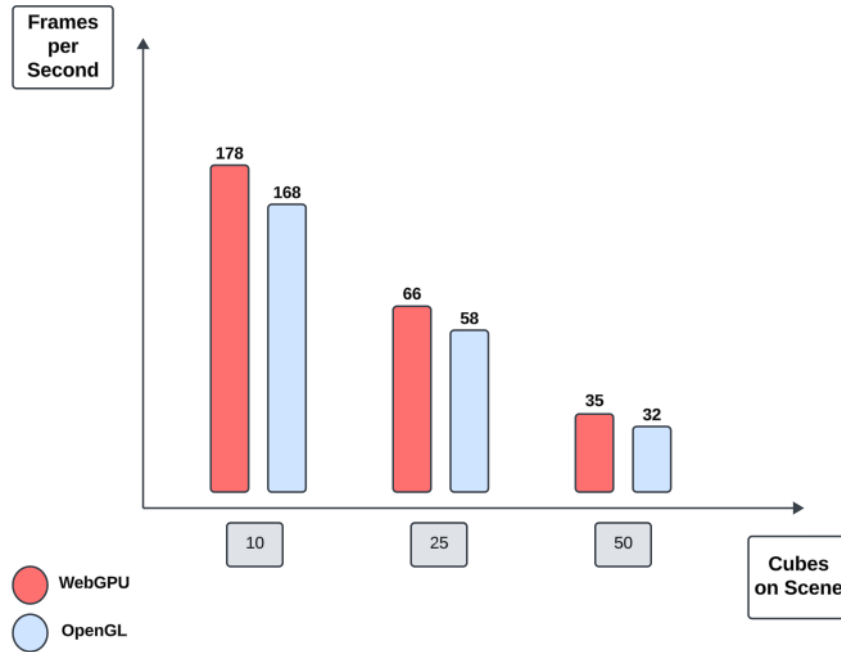


Figure 38: Performance metrics recorded on a ryzen 7 processor, and an AMD 7900 xtx graphics card.

Both the WebGPU and OpenGL renderers used the same shaders and configurations for the skybox and cube rendering. The key difference between the two implementations lies in the underlying API and the rendering architecture. As we can see from *Figure 38* WebGPU renderer was constantly about 3 to 10 fps higher from the OpenGL one. That's just goes to show that the CPU was much more of a limiting factor than the GPU, there for there is a lot of room for performance gains in the future.

Moreover, WebGPU allows for more fine-grained control over resource management, including buffers, textures, and memory access, resulting in a more

efficient rendering pipeline. Even with the additional CPU demands from the subsystem-based architecture, the improved GPU performance leads to higher frame rates, faster processing of complex scenes, and better overall scalability compared to OpenGL.

However, it is important to note that examples with higher subsystem activation, such as SSAO usage, exhibit a significant performance decrease (e.g. in *Figure 39* the scene is loaded with a relatively complex mesh and also uses SSAO, we can see a significant performance decrease in comparison with *Figure 38* where in that test no more than 3 sub-systems were active at the same time). This is to be expected, as these additional subsystems introduce more computational overhead, particularly on the CPU, due to the increased number of calculations and memory access required. While the overall performance remains superior to OpenGL, more complex scenes involving multiple subsystems will naturally have a larger impact on frame rates and performance.



Figure 39: Example of the performance decrease due to the high usage of the rendering subsystems in a relatively complex scene with SSAO and other subsystems active.

Despite this, the shift to WebGPU allows for better multi-threading and asynchronous task execution, which helps to alleviate CPU bottlenecks and fully utilize modern hardware capabilities. This results in a more balanced workload distribution between the CPU and GPU, ultimately leading to performance gains in both rendering speed and visual quality.

In summary, while the WebGPU renderer introduces a more CPU-intensive system, the improved API design and GPU optimizations translate into significant performance benefits over the OpenGL implementation. The occasional performance drops with heavy subsystem usage are counterbalanced by the overall efficiency and flexibility of the WebGPU pipeline.

6 Examples

6.1 Introduction Example

In this example, we will render a simple cube using the forward rendering system within the framework. The cube will be displayed in a scene with a moving camera, demonstrating the use of mesh, material, and shader components, along with the application of WGSL shaders. The forward rendering pipeline will be utilized to produce the final image of the cube in the scene.

To set up the scene (*Figure 40*), we begin by importing and assigning the necessary components to their corresponding entities. The camera entity, for instance, is equipped with a camera component, a camera movement component, and a transform component to manage its position and movement within the scene. The cube entity is assigned a mesh component, which defines its geometry, along with a material component to handle its appearance, and a transform component to control its placement and orientation. Additionally, the cube entity includes a forward rendering shader component to specify the shaders required for the vertex and fragment stages during the rendering process. These components collectively allow the system to render the cube in the scene as the camera moves around it, demonstrating the interaction of entities within the forward rendering system.

```
camera = Scene().add_entity()
Scene().add_component(camera, InfoComponent("main camera"))
Scene().add_component(camera, TransformComponent(glm.vec3(0, 0, 10), glm.vec3(0, 0, 0), glm.vec3(1, 1, 1), static=False))
Scene().add_component(camera, CameraComponent(60, 16/9, 0.01, 500, 10, CameraComponent.Type.PERSPECTIVE))
Scene().add_component(camera, CameraControllerComponent())
Scene().set_primary_cam(camera)

model = Scene().add_entity()
Scene().add_component(model, InfoComponent("model"))
Scene().add_component(model, TransformComponent(glm.vec3(0, 0, 0), glm.vec3(0, 0, 0), glm.vec3(1, 1, 1), static=True))
Scene().add_component(model, MeshComponent(mesh_type=MeshComponent.Type.IMPORT, import_path=definitions.MODEL_DIR / "cube" / "source" / "cube.obj"))
Scene().add_component(model, ForwardShaderComponent(shader_path=definitions.SHADER_DIR / "WGPU" / "base_color_shader.wgsl"))
Scene().add_component(model, MaterialComponent())
```

Figure 40: Example of adding entities and components in the Scene.

Furthermore, we need to add the appropriate systems to manage the components within the scene (*Figure 41*). Each system will have corresponding filters that dictate which entities the system will act upon. For instance, the camera movement system will filter for entities with both a camera component and a transform component, allowing it to update the camera's position and orientation based on user input. Similarly, the rendering system will filter for entities with mesh, material, and shader components, ensuring that only entities with all the necessary components are processed during the rendering stage. By setting up these systems with their corresponding filters, we ensure that each system operates only on the relevant entities, efficiently managing the scene's logic and rendering.

Finally, we instantiate the renderer, which handles the actual drawing of the

```

Scene().add_system(SkyboxSystem([SkyboxComponent]))
Scene().add_system(TransformSystem([TransformComponent]))
Scene().add_system(CameraSystem([CameraComponent, TransformComponent]))
Scene().add_system(CameraControllerSystem([CameraControllerComponent, CameraComponent, TransformComponent]))
Scene().add_system(MeshSystem([MeshComponent]))
Scene().add_system(DeferredLightShaderSystem([DeferredLightComponent]))
Scene().add_system(ForwardShaderSystem([ForwardShaderComponent]))

```

Figure 41: Example of adding systems and filters in the Scene.

scene. In the main function, we update the uniforms with any dynamic data, such as the camera's position or lighting parameters, ensuring that the shaders receive the most current information. Once the uniforms are updated, we call the render function, passing in the desired resolution for the final output (*Figure 42*). This triggers the rendering pipeline, processing all the components and systems, and ultimately producing the rendered image on the screen.

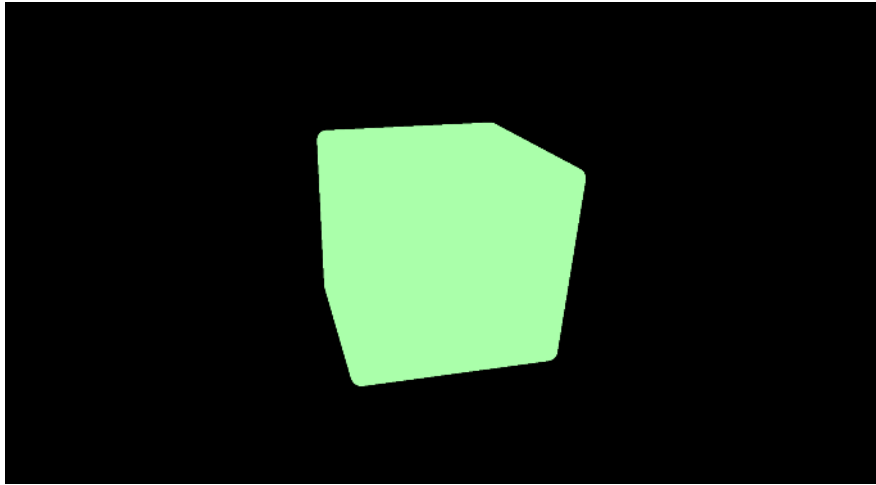


Figure 42: A simple example using the WebGPU renderer of Elements.

6.2 Textures Example

To add a textured mesh into the scene, we first bake the texture into the `TextureLib` (*Figure 43*), making it accessible within our framework.

```

TextureLib().make_texture(name="grass", path=definitions.TEXTURE_DIR / "Texture_Grass.png")

```

Figure 43: Example of baking a texture for later usage.

Once the texture is stored, we use the GPU controller to manually send the texture and its corresponding sampler to the shader via the `set_texture_sampler` function (*Figure 44*). This method allows for precise control over how textures are applied to the mesh during rendering, ensuring that the texture is correctly sampled and displayed on the object within the scene. This process integrates smoothly with our existing systems, providing flexibility in texture management.

```

GpuController().set_texture_sampler(
    shader_component=Scene().get_component(model, ForwardShaderComponent),
    texture_name="diffuse_texture",
    sampler_name="diffuse_sampler",
    texture=TextureLib().get_texture(name="grass")
)

```

Figure 44: Example of how to set the texture and the sampler for a shader component.

Finally, we proceed as usual by updating the uniform variables of the shader. With the texture and sampler now set up in the `TextureLib`, we can reference them directly within the shader using the names we've assigned. This allows us to produce a mesh with the texture properly applied (*Figure 45*), ensuring that the texture maps correctly onto the surface of the object during rendering.

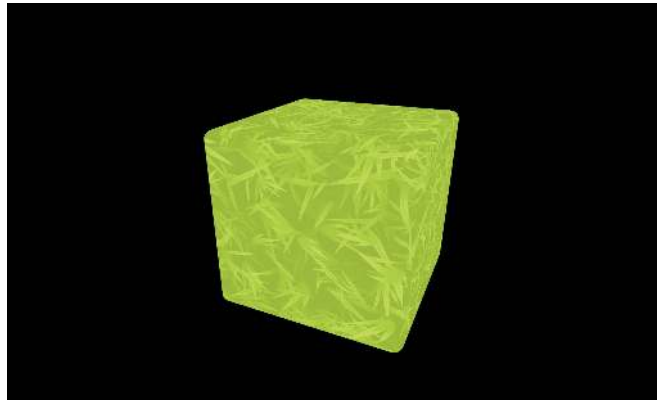


Figure 45: A scene with a mesh mapped with a texture.

6.3 Lighting

In this example, we build upon the introductory setup by adding a light source to the scene (*Figure 46*). The object in the scene will be rendered using a Blinn-Phong shader, which is a basic lighting model that simulates how light interacts with the surface of an object. This model is particularly effective for creating realistic specular highlights and diffuse lighting, making it ideal for simple but visually appealing lighting effects.

```

light = Scene().add_entity()
Scene().add_component(light, InfoComponent("model"))
Scene().add_component(light, TransformComponent(glm.vec3(5, -10, 5), glm.vec3(0, 0, 0), glm.vec3(0.3, 0.3, 0.3), static=True))
Scene().add_component(light, MeshComponent(mesh_type=MeshComponent.Type.IMPORT, import_path=definitions.MODEL_DIR / "cube" / "source" / "cube.obj"))
Scene().add_component(light, ForwardShaderComponent(shader_path=definitions.SHADER_DIR / "WGPU" / "base_color_shader.wgsl"))
Scene().add_component(light, LightComponent(intensity=1.0, color=glm.vec3(1.0, 1.0, 0.5)))
Scene().add_component(light, MaterialComponent())

```

Figure 46: Example of adding a light entity with the necessary components.

To implement this, we'll introduce a light entity to the scene. Additionally, we add the `LightAffection` component to the entity representing the mesh. This component establishes a link between the mesh and the specific light source. Additionally we adjust the shaders to account for the light source, and update

the rendering process to incorporate the Blinn-Phong shading calculations (*Figure 47*). The rest of the setup remains similar to the previous example, but with the added complexity of lighting, which enhances the overall realism of the scene.

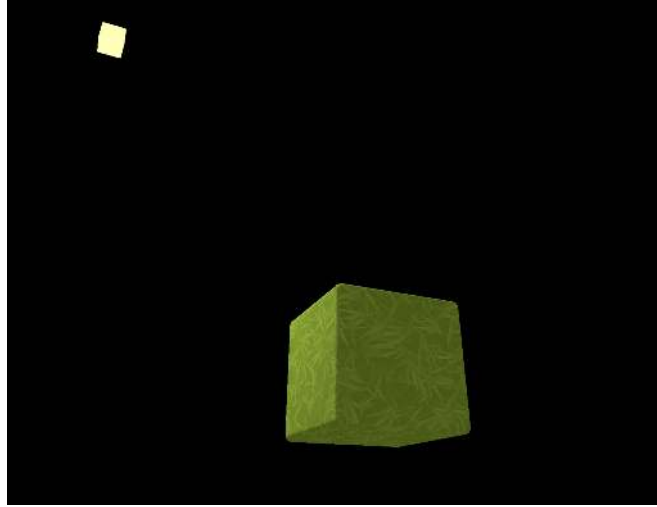


Figure 47: Scene with a mesh using a Blinn-Phong shader and a light source.

6.4 Skybox

Building upon the introductory example, we now introduce a skybox to the scene. To achieve this, we add a **SkyboxSystem** to the existing setup. This system will be responsible for rendering the skybox, providing a background that envelops the entire scene. Alongside this, we create a **SkyboxEntity** that includes the necessary **SkyboxComponent** (*Figure 48*). That component defines the textures for the skybox, allowing it to seamlessly blend with the scene's environment.

```
skyPaths = [
    definitions.TEXTURE_DIR / "Skyboxes" / "Sea" / "back.jpg",
    definitions.TEXTURE_DIR / "Skyboxes" / "Sea" / "front.jpg",
    definitions.TEXTURE_DIR / "Skyboxes" / "Sea" / "bottom.jpg",
    definitions.TEXTURE_DIR / "Skyboxes" / "Sea" / "top.jpg",
    definitions.TEXTURE_DIR / "Skyboxes" / "Sea" / "right.jpg",
    definitions.TEXTURE_DIR / "Skyboxes" / "Sea" / "left.jpg",
]

sky = Scene().add_entity()
Scene().add_component(sky, InfoComponent("cubemap"))
Scene().add_component(sky, SkyboxComponent("sky", skyPaths))
```

Figure 48: Gathering the paths of the skybox textures and insert them to the skyboxComponent.

The skybox rendering process is handled within a dedicated rendering subsystem. Unlike other shaders that might vary depending on the specific needs of each application, the shader used for skyboxes is hard-coded since it remains consistent across different implementations. This simplifies the process, as all that's required is to bake the textures for the skybox and that's being handled from the skybox system (*Figure 49*). Once these textures are prepared, the rendering

subsystem automatically handles the rest, applying the skybox shader to create the immersive background environment in the scene.

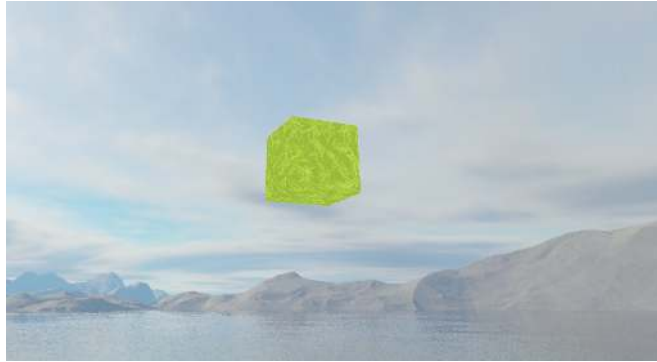


Figure 49: Example of a scene with a skybox.

6.5 Normal Mapping

Normal mapping is a technique used in 3D graphics to add detailed surface texture to a model without increasing its polygon count. It achieves this by altering the way light interacts with the surface, giving the illusion of complex bumps and wrinkles. Instead of modifying the actual geometry, a normal map (*Figure 50*)—a special texture that contains information about the surface normals—affects how light is reflected, creating the appearance of depth and detail.

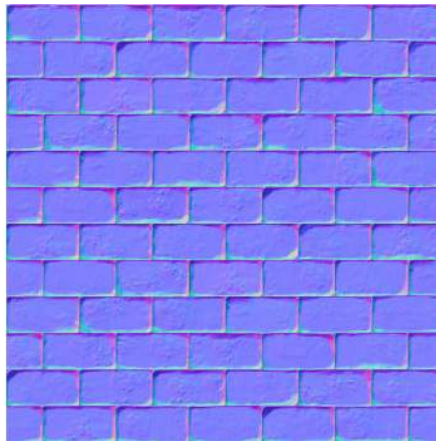


Figure 50: Example of a normal texture.

To integrate normal mapping into our scene, we build upon the basic setup from the intro and texture examples. After baking the normal map texture into the `TextureLib`, we send it to the GPU through the `set_texture_sampler` function, just like with the diffuse texture. The shader then uses this normal map to modify the lighting calculations (*Figure 51*), resulting in a more detailed and realistic rendering of the mesh surfaces, enhancing the visual richness of the scene without adding extra geometry.

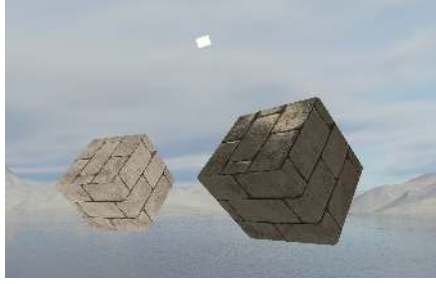


Figure 51: Example of a Scene Pointing Out the Difference Between Normal Mapping and Simple Coloring.

6.6 Parallax Occlusion Mapping

Parallax Occlusion Mapping (POM)[8] is an advanced texturing technique used to simulate intricate surface detail and depth on textures, enhancing the visual complexity of a surface without increasing the actual geometry. Unlike traditional normal mapping, which modifies the surface appearance by altering surface normals, POM creates the illusion of depth by displacing texture coordinates based on a height map (*Figure 52*). This technique is particularly effective for adding realistic depth effects to surfaces, making them appear more detailed and dynamic.

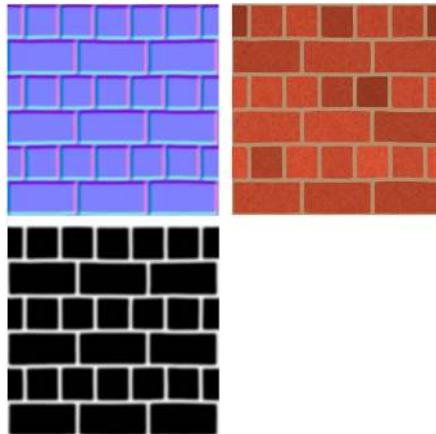


Figure 52: Textures used in the Parallax Occlusion Mapping example.

In the context of integrating Parallax Occlusion Mapping into a scene, the process involves several key steps. Building upon the previous examples, we start by preparing a height map texture along with a standard color texture for the surface. This height map will be used to calculate the displacement for the POM effect.

The integration involves updating the shader to handle POM calculations. This means modifying the fragment shader to incorporate the height map and adjust the texture sampling coordinates based on the height information. The texture and height map are baked into the ‘TextureLib’ and then bound to the shader via the GPU controller. Finally, we update the uniform variables of the

shader to include parameters related to POM, allowing the shader to render surfaces with a convincing sense of depth (*Figure 53*).

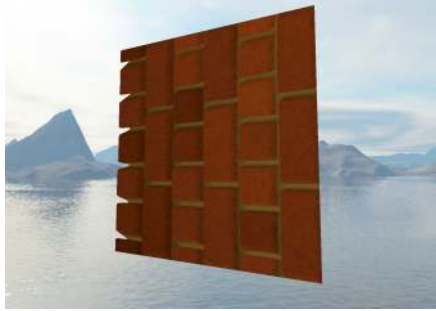


Figure 53: Example of a scene with an entity with Parallax Occlusion Mapping.

This approach enhances the realism of textures in the scene, providing a more detailed appearance compared to simple coloring or even normal mapping.

6.7 Shadows

Shadow casting in a rendering pipeline involves simulating the way light interacts with objects to create shadows, adding realism and depth to a scene. In a typical scenario, shadow casting is achieved by rendering the scene from the perspective of the light source to generate a depth map, often referred to as a shadow map. This depth map stores the distance of the nearest object from the light source for each pixel.

During the rendering pass for the scene, each fragment's position is compared to the shadow map. If the fragment is farther from the light source than the depth recorded in the shadow map, it is considered to be in shadow. This comparison allows the renderer to accurately determine which areas of the scene should be darkened, simulating the effect of shadows being cast by objects blocking the light.

In our framework, implementing shadow casting involves creating a dedicated rendering subsystem that first generates the shadow map. This is done by rendering the scene from the light's point of view into a texture, storing the necessary depth information. Then, during the main rendering pass, this shadow map is used to calculate whether each fragment should be lit or shadowed, based on its position relative to the light source and the objects blocking it.

To enable shadow casting, we need to add a `CameraComponent` to the light source (*Figure 54*), as the shadow rendering pass filters based on the presence of both a `LightComponent` and a `CameraComponent`.

```

light = Scene().add_entity()
Scene().add_component(light, InfoComponent("light"))
Scene().add_component(light, TransformComponent(glm.vec3(0, -10, 10), glm.vec3(30, 0, 0), glm.vec3(1, 1, 1), static=False))
Scene().add_component(light, LightComponent(intensity=1.0))
Scene().add_component(light, CameraComponent(60, 16/9, 0.01, 500, 35, CameraComponent.Type.PERSPECTIVE))

```

Figure 54: Adding a camera component to the light entity.

Any entity that should cast or receive shadows must be assigned with a `ShadowAffectedComponent` (Figure 55). Once this is set up, the shadow rendering system will include all entities that meet these criteria in the shadow mapping process.

```

building = Scene().add_entity()
Scene().add_component(building, InfoComponent("building"))
Scene().add_component(building, TransformComponent(glm.vec3(0, 2, -5), glm.vec3(0, -90, 180), glm.vec3(0.01, 0.01, 0.01), static=True))
Scene().add_component(building, MeshComponent(mesh_type=MeshComponent.Type.IMPORT, import_path=definitions.MODEL_DIR / "stronghold" / "source" / "building.obj"))
Scene().add_component(building, ForwardShaderComponent(shader_path=definitions.SHADER_DIR / "MGPU" / "blin_phong_shadow_shader.wgsl"))
Scene().add_component(building, MaterialComponent())
Scene().add_component(building, ShadowAffectionComponent(light_entity=light))

```

Figure 55: Attaching a `ShadowAffectedComponent` to the correspondent affected entity.

Finally, in the coloring shader of each entity, the generated shadow map can be accessed using `shadow_texture`, and the corresponding sampler with `shadow_sampler`. This allows the shader to incorporate shadows into the final rendering (Figure 56), creating a more realistic scene.



Figure 56: Example of a scene with an entity with a shadow.

6.8 Screen Space Ambient Occlusion

Screen Space Ambient Occlusion (SSAO) is a shading technique that enhances the realism of a scene by simulating how ambient light is occluded by nearby objects, creating subtle shadows in crevices and around objects. SSAO operates in screen space, using the data from the G-buffer—specifically, the depth and normal textures—to calculate occlusion.

In this example, SSAO is integrated into the deferred rendering system. The process involves two stages: first, calculating the occlusion factor for each pixel based on the depth and normal information from the G-buffer; second, applying a blur to the resulting SSAO texture to smooth out any noise. This final SSAO texture is then used during the lighting pass to modulate the ambient light, adding depth and realism to the scene.

```
Renderer().init(  
    present_context=present_context,  
    render_texture_format=render_texture_format,  
    SSAO=True  
)
```

Figure 57: Setting the SSAO parameter to true in order to enable the SSAO rendering sub-system.

To enable SSAO in this iteration, you must notify the renderer by setting the `SSAO` parameter to `True` (*Figure 57*) during the renderer initialization. Once enabled, the renderer will produce the SSAO texture, which will then be available in the coloring shader for use in enhancing the visual realism (*Figure 58*) of the scene through ambient occlusion effects.



Figure 58: Figure capturing the difference between real time rendering with SSAO and without SSAO.

References

- [1] Alan Baker, Mehmet Oguz Derin, and David Neto. Shading language for webgpu. <https://www.w3.org/TR/WGSL/>.
- [2] Louis Bavoil and Miguel Sainz. Screen space ambient occlusion. *NVIDIA developer information*: <http://developers.nvidia.com>, 6(2), 2008.
- [3] Marcus Geelnard and Camilla Löwy. Glfw is an open source, multi-platform library for opengl, opengl es and vulkan development on the desktop. it provides a simple api for creating windows, contexts and surfaces, receiving input and events. <https://www.glfw.org/>.
- [4] Yousra J'lali. Directx 12: Performance comparison between single-and multithreaded rendering when culling multiple lights, 2020.
- [5] John Kessenich, Graham Sellers, and Dave Shreiner. *OpenGL Programming Guide: The official guide to learning OpenGL, version 4.5 with SPIR-V*. Addison-Wesley Professional, 2016.
- [6] Elie Michel. Webgpu c++ guide. <https://eliemichel.github.io/LearnWebGPU/getting-started/hello-webgpu.html>.
- [7] George Papagiannakis, Manos Kamarianakis, Antonis Protopsaltis, Dimitris Angelis, and Paul Zikas. Project elements: A computational entity-component-system in a scene-graph pythonic framework, for a neural, geometric computer graphics curriculum. *arXiv preprint arXiv:2302.07691*, 2023.
- [8] Natalya Tatarchuk. Dynamic parallax occlusion mapping with approximate soft shadows. In *Proceedings of the 2006 symposium on Interactive 3D graphics and games*, pages 63–69, 2006.
- [9] Jonathan Thaler and TU Wien. Deferred rendering. *TU Wien: Vienna, Austria*, 2011.
- [10] Amund Tveit, Torbjørn Morland, and Thomas Brox Røst. Deeplearningkit- an open source deep learning framework for apple’s ios, os x and tvos developed in metal and swift.
- [11] Kevin Ubay-Ubay. Real time rendering engine. *Computers & Graphics*, 2013.

- [12] Johannes Unterguggenberger, Bernhard Kerbl, and Michael Wimmer. Vulkan all the way: Transitioning to a modern low-level graphics api in academia. *Computers & Graphics*, 111:155–165, 2023.
- [13] Marcus von Appen. Pysdl2 is a wrapper around the sdl2 library and as such similar to the discontinued pysdl project. in contrast to pysdl, it has no licensing restrictions, nor does it rely on c code, but uses ctypes instead. <https://pysdl2.readthedocs.io/en/0.9.13/>.
- [14] Jiahao Zou, Zhen Xu, Sheng Wu, and Lin Zhou. Improved anti-aliasing algorithm based on fast approximate anti-aliasing. In *Fifteenth International Conference on Graphics and Image Processing (ICGIP 2023)*, volume 13089, pages 568–579. SPIE, 2024.